

飞书文档 -> 向量数据库 -> 问答流程说明

- **第 1 步的“发送”动作：**当用户在飞书聊天框发消息时，飞书服务器把这个事件推送到你配置的 URL（即 /api/feishu/events）。

订阅方式 [🔗](#)

将事件发送至 **开发者服务器**

请求地址 [?](#)

`https://711b699b.r28.cpolar.top/api/feishu/events`

需要是外网地址

- **第 2 步 (FeishuEventService)：**后端接收到飞书发来的 JSON 报文，由这个 Service 进行初步解析（比如验签、解密、识别事件类型）。

1：响应 URL 验证 (Challenge)

防误配：你把 URL 填错（如多一个字符），飞书不会开始推送事件到错误地址。飞书在“配置/变更事件订阅地址”时，发一次握手请求；原样回 challenge，表示“这个 URL 是活的、

2：安全验证 (Security Check) —— 核心前置

在处理任何逻辑之前，它先做了两道安全检查，确保请求真的来自飞书，而不是黑客伪造的：

- **verification token**: 检查 token 是否匹配。证明是自己的飞书发的，而不是黑客发的。
- **Encrypt Key**: 通过 timestamp、nonce 和 encrypt_key 进行哈希校验，验证报文是否被篡改。（这个没配置）

加密策略

用于加密事件或回调的请求内容，校验请求来源。当订阅方式为“[将事件发送至开发者服务器](#)”或“[将回调发送至开发者服务器](#)”时生效

Encrypt Key

未开启，请点击[重置](#)或进行[自定义编辑](#) [C](#) [E](#)

Verification Token

***** [E](#) [C](#) [E](#)

3：事件分发

- 第一种走bot对话处理

```
{
  "header": {"event_type":
    "im.message.receive_v1"},
  "event": {
    "message": {
      "message_id": "om_xxx",
      "message_type": "text",
      "content": "{\"text\":\"请索引这个
https://xxx.feishu.cn/docx/doxabc123\"}"
    }
  }
}
```

已添加事件

添加事件

事件名称	订阅类型 ①	所需权限 (开通以下任一权限即可)	操作
接收消息 v2.0 im.message.receive_v1	应用身份	- 读取用户发给机器人的单聊消息 已开通 - 接收群聊中@机器人消息事件 已开通 - 获取群组中所有消息 (敏感权限) 已开通	删除事件

○ 第二种和第三种

payload 扫描 doc/wiki token, 走 文档索引链路。
例子 (会提取到 wik..., 进入索引分支) :

```
{
  "header": {"event_type": "drive.file.updated_v1"},
  "event": {
    "file_url": "https://xxx.feishu.cn/wiki/wik9Y8Z7",
    "title": "产品方案"
  }
}
```

第三条：在“文档链路”里再二次分流：删除事件走删除，其他走新增/更新索引。

例子（会走 delete_source）：

```
{  
  "header": {"event_type": "drive.file.deleted_v1"},  
  "event": {  
    "url": "https://xxx.feishu.cn/docx/doxDEL123",  
    "is_deleted": true  
  }  
}
```

} 都没有实现，也没有订阅这个事件，如果扩展可以，实现不用@机器人，直接在对应的知识库里修改，并配置到本地数据库

• 第 3 步 FeishuBotService :

1：异步处理：

- 收到后立刻把 解析文本 -> 提取链接 -> 索引文档 -> 回复用户。丢进后台队列
- 先回复{"ok": true}（配合 2xx）= 明确告诉飞书“我收到了，不用重试”。

2：后台线程，开始慢慢做事情 后台队列里的事情

从文本提取 token

- 依次提取 docx、legacy doc、wiki token（正则匹配链接或裸 token）。
- 三种都没有，就 `reply_text("No document link found...")` 结束。
`app/services/feishu_bot.py:50-58,`
`app/services/feishu_bot.py:98-122`

调索引器拉文档并入库

- 有 docx -> `index_source("docx", token, "")`
- 否则 wiki -> `index_source("wiki", token, "")`
`app/services/feishu_bot.py:60-67`

`index_source` 内部做的事

- 调飞书 API 拉原文内容（docx/doc/wiki）。
- 生成 `doc_id = "{source_type}:{token}"`。
- 交给向量库：文本归一化 -> 截断 -> 分块 -> 删除旧块。
`app/services/feishu_doc_indexer.py:30-44,`
`app/services/feishu_doc_indexer.py:52-69,`
`app/services/vector_store.py:50-80`
- AI向量化后，进行写入新块

- 它使用的是一个预先训练好的神经网络模型（在你这里是 bge-small-zh-v1.5）。

这个模型在“上学”期间读过了几乎整个互联网的文本，它知道“西红柿”和“番茄”在语义上是非常接近的。

- **AI 的作用：**它不是简单地给字符编码，而是提取文字背后的“**意思**”，并把这个“意思”转化成几百个精确的坐标数值。给用户回结果

- 假设你有一句话：“如何报销差旅费？”

经过 AI 向量化后，它会变成一个长度为 512 或 768 的数字列表（数组）：

[0.12, -0.05, 0.88, 0.23, ...]

- 任何异常：reply_text("Index failed: ...")
- 成功：reply_text("Document indexed. **You can ask now.**")

app/services/feishu_bot.py:67-71



• 第4步提问:

检索

- 输入: 用户原始问题 message
- 调 Chroma 查询: `collection.query(query_texts=[query], n_results=top_k)`
(app/services/vector_store.py:95-98)
- 这里 Chroma 会用同一个 embedding 模型把 query 向量化 (你配置的 FEISHU_KB_EMBED_MODEL, 默认 BAAI/bge-small-zh- v1.5)

- 与库里 chunk 向量做相似度检索，返回 topK 候选（默认 FEISHU_KB_TOP_K=6）
- 结果后处理：score = 1 - distance，低于 FEISHU_KB_MIN_SCORE（默认 0.2）的丢掉（偏离太远的）
- 生成
- 把阶段1拿到的 kb_results 拼进 prompt（知识库检索结果：1. 来源[...] 内容...）
- 同时拼入实时指标/趋势/洞察，形成完整 user_prompt
- 用 system_prompt 约束模型“优先用知识库，不要编造”
- 调 DeepSeekClient.chat() 生成最终回答（app/services/chat.py:41）

补充说明： 机器人权限，具体机器人做事需要开启对应的权限

权限管理

开通 API 权限后，应用才能以应用身份 (tenant_access_token) 或用户身份 (user_access_token) 调用飞书 API；以应用身份调用飞书 API 时，应用可能还需要申请对应的数据权限。[了解更多](#)

[收起介绍](#)



开通权限

批量导入/导出权限

Q 例如：获取群组信息、im:cha...

权限名称

业务模块

权限类型

权限状态

权限名称	权限类型	权限状态	可访问的数据范围	配置	操作
获取文件 aily:file:read	应用身份	已开通	-		相关 API/事件 关闭
获取消息 aily:message:read	应用身份	已开通	-		相关 API/事件 关闭
发送消息 aily:message:write	应用身份	已开通	-		相关 API/事件 关闭